Week 6 Handout

Training Classical ML Models — Model Report Lab

This handout supports Week 5-6 class sessions (the supervised learning workflow; linear regression with Ridge/Lasso; logistic regression, decision trees, and k-nearest neighbors; and confusion-matrix/precision/recall/AUC evaluation). Read this whole handout before you start — Part C shows a complete worked example end to end, and Part D gives you starter code for all four model types so you're not writing anything from a blank page.

Part A — Diagnose Your Problem

Step 1 — Choose ONE of the two datasets below, load it with the code shown, and identify the target column — the thing you're trying to predict. State in one sentence whether it's a continuous number (→ regression) or a category (→ classification). Both datasets ship with scikit-learn, so there is nothing to download or upload — they load in Colab or any local Python install.

Dataset	What it contains	Target (y)	Size
Diabetes progression (load_diabetes)	10 baseline health measurements per patient (age, sex, BMI, blood pressure, and six blood-serum values), already centered and scaled by the dataset's authors.	A number measuring disease progression one year later	442 patients
Breast cancer diagnosis (load_breast_cancer)	30 measurements computed from images of a cell sample (e.g., radius, texture, smoothness), on very different numeric ranges.	0 = malignant, 1 = benign	569 samples

```
# Option A — Diabetes progression
from sklearn.datasets import load_diabetes
data = load_diabetes(as_frame=True)

# Option B — Breast cancer diagnosis
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer(as_frame=True)

X, y = data.data, data.target   # features table, target column
```

Run only the option you chose (the two blocks both define data). X is a pandas table of features and y is the target — these are the X and y used in the rest of this handout.

How to tell the difference

- Continuous number: price, temperature, age, a test score, time until an event — anything you could plot on a number line and it would make sense to say “14.7” or “-3.2.”
- Category: yes/no, spam/not-spam, a product category, a risk tier (low/medium/high) — a fixed, named set of possible answers.
- When in doubt, ask: “could the true answer be exactly 14.7?” If yes, it's regression. If the answer has to be one of a short list of labels, it's classification.

Part B — Preprocess (Recap from Week 5)

Step 2 — Inspect your data and apply the Week 5 steps that don't depend on the train/test split: check for missing values (X.isna().sum()) and look for categorical columns (X.dtypes). Scaling comes later, in Step 4, after the split. Not every dataset needs every step — decide based on what X.isna().sum() and X.dtypes actually show you, and say in your report what you checked and what you decided. (Hint: look closely at both datasets before deciding what, if anything, needs doing.)

One thing that's different this week

- Decision trees don't need scaled features — splits like "age > 40" work identically whether age is in years or scaled to mean 0. Scaling before a tree isn't wrong, it's just unnecessary work.
- Logistic regression, k-nearest neighbors, AND Ridge/Lasso all DO benefit from scaled numeric features when you have more than one — KNN especially, since it's distance-based and an unscaled feature on a large numeric range can silently dominate the distance calculation. Ordinary LinearRegression gives the same predictions and R^2 either way, so it doesn't need scaling.
- Order matters: split first, then scale. Anything that learns from the data — a scaler's mean and standard deviation, or a median used to fill blanks — must be computed from the training rows only, then applied to the test rows. Fitting it on the full dataset lets test-set information leak into training (the Week 5 data-leakage rule).

Part C — Split, Fit, Predict, Evaluate

Step 3 — Split your data with train_test_split(X, y, test_size=0.2, random_state=42) — use exactly random_state=42 so your results are comparable to your classmates' and to the answer key.

Step 4 — If you're using logistic regression, k-nearest neighbors, or Ridge/Lasso with more than one numeric feature, scale now: fit a StandardScaler on X_train only, then transform both X_train and X_test (starter code in Part D). Skip this step for decision trees and plain LinearRegression.

Step 5 — Fit an appropriate model on the training set, predict on the test set, and report the test-set score: model.score(X_test, y_test) gives you R^2 for a regressor or accuracy for a classifier. If your target is a category, also report precision and recall (sklearn.metrics.classification_report) — not accuracy alone, per Day 4's evaluation lecture.

WORKED EXAMPLE **A Complete Model Report (House Price → Linear Regression)**

Target & type: "price" — a continuous dollar amount, so this is a regression problem.

Preprocessing: "No missing values in this dataset (df.isna().sum() was all zero); sqft was already numeric and the model has only one feature, so no encoding or scaling was needed."

Split & fit: "Split 80/20 with random_state=42 (8 training rows, 2 test rows), then fit sklearn.linear_model.LinearRegression() on the training set."

Score: "R^2 on the test set was 0.9993 — the model explains almost all of the price variation from square footage alone."

What it got wrong: "The model predicted $260,577 for a house that actually sold for $260,000 — a $577 overshoot. My best guess is this house had some feature not in the data (like a smaller lot or older roof) that slightly lowered its real price below what square footage alone would predict."

Part D — Optional: Starter Code for All Four Models

This part isn't graded — it's here so you're adapting working code to your dataset instead of writing it from scratch. Pick the block that matches your Part A diagnosis.

If your target is a continuous number:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
print("R^2 on test set:", model.score(X_test, y_test))
```

Swap X and y for your own feature(s) and target column.

If you need scaling (logistic regression, KNN, Ridge/Lasso) — do this after the split:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)  # learn mean/std from TRAIN only
X_test_s = scaler.transform(X_test)        # reuse them on test — never refit
```

Use X_train_s / X_test_s in the blocks below that say "scaled"; the decision tree can keep using X_train / X_test. (If you chose the diabetes data, its features were already scaled by the dataset's authors, so this step changes little there.)

Optional: regularized regression (continuous target, more than one feature):

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0).fit(X_train_s, y_train)
lasso = Lasso(alpha=1.0).fit(X_train_s, y_train)
print("Ridge R^2:", ridge.score(X_test_s, y_test))
print("Lasso R^2:", lasso.score(X_test_s, y_test))
```

Compare against plain LinearRegression's R^2 — and check lasso.coef_ to see whether any feature was set to zero.

If your target is a category and you want a probability:

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(X_train_s, y_train)
print("Accuracy on test set:", clf.score(X_test_s, y_test))
print("P(class 1):", clf.predict_proba(X_test_s)[:, 1].round(2))
```

Uses the scaled X_train_s / X_test_s from the scaling block above, with the same y_train / y_test.

If your target is a category and you want an explainable flowchart:

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
print("Accuracy on test set:", tree.score(X_test, y_test))
print(dict(zip(X.columns, tree.feature_importances_.round(2))))
```

max_depth=3 is a reasonable starting point — raise it if the model underfits, lower it if training accuracy looks suspiciously perfect.

If your target is a category and you want a similarity-based model (scaled features):

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_s, y_train)
print("Accuracy on test set:", knn.score(X_test_s, y_test))
```

n_neighbors=5 is a reasonable starting point — use the scaled data (Step 4) or this model's distances won't be meaningful.

Optional: evaluate a classifier beyond accuracy (Day 4)

```
from sklearn.metrics import (
    confusion_matrix, classification_report, roc_auc_score
)

# For the tree, use tree.predict(X_test) instead
preds = clf.predict(X_test_s)
print(confusion_matrix(y_test, preds))
print(classification_report(y_test, preds))

# Optional, two-class targets only:
print("AUC:", roc_auc_score(y_test, clf.predict_proba(X_test_s)[:, 1]))
```

Swap in whichever classifier you fit above, using the matching test set (scaled for logistic regression and KNN, unscaled for the tree).

Step 6 — Submit

Write up your Part A diagnosis, Part B preprocessing notes (including whether and why you scaled), and Part C split/fit/score results, plus the "what it got wrong" sentence from the worked example above, applied to YOUR dataset. Submit as a single 1-page document (code + written answers) as a PDF or DOCX through the course platform before the next class.

Common pitfalls

- Reporting a score with no interpretation — "$R^2 = 0.87$" alone earns partial credit; say whether that's good and what it means for this problem.
- Reporting only accuracy for a classification target — a class-imbalanced dataset can make a useless model look great; also report precision and recall (Day 4).
- Using a different random_state than 42 — your split (and therefore your score) won't match what the grader expects when checking your work.
- Picking a model that doesn't match your target type — a category target needs logistic regression, a decision tree, or k-nearest neighbors, never LinearRegression.
- Skipping the "what did it get wrong" question — this is worth real credit and is usually the most interesting part of the report to write.

What to Submit

Item	Format	Notes
Model report (diagnosis + preprocessing + split/fit/score + error analysis)	PDF or DOCX (code + ~200–300 words)	Single combined 1-page document

Grading Rubric

Criterion	Weight	Description
Correct problem diagnosis	20%	Correctly identifies target type (number vs. category) and picks a matching model family.
Preprocessing applied correctly	20%	Applies only the Week 5 steps the dataset actually needs, and briefly explains the choice.
Split, fit, and score executed correctly	25%	Uses train_test_split with random_state=42, fits on training data only, scores on the held-out test set.
Interpretation of the score	20%	States whether the score is good for this problem, not just the bare number. For a classification target, includes precision/recall, not accuracy alone.
Error analysis	15%	Identifies one specific test-set case the model got wrong and gives a plausible reason why.

Resources

scikit-learn documentation for this week's four models:

- train_test_split — splitting data into training and test sets: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- LinearRegression, Ridge, Lasso — fitting and scoring a regression model, with and without regularization: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- LogisticRegression — fitting a classifier and reading predict_proba: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- DecisionTreeClassifier — fitting a tree and reading feature_importances_: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
- KNeighborsClassifier — fitting a k-nearest neighbors classifier: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html
- confusion_matrix, classification_report, roc_auc_score — evaluating a classifier beyond accuracy (Day 4): https://scikit-learn.org/stable/modules/model_evaluation.html
- The two datasets — descriptions of every column: https://scikit-learn.org/stable/datasets/toy_dataset.html

Background reading on this week's concepts:

- scikit-learn's own guide to under/overfitting with a worked example: https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html
- Week 5 handout (for the preprocessing steps this week assumes you already know).